

Model-Based Framework for Continuous Adaptation and Evolution of Quantum-Classical Hybrid Systems

Fuyuki Ishikawa

f-ishikawa@nii.ac.jp

National Institute of Informatics
Chiyoda-ku, Tokyo, Japan

Shinobu Saito

shinobu.saito@ntt.com

NTT Computer and Data Science Laboratories
Musashino-shi, Tokyo, Japan

ABSTRACT

Although quantum computing has been attracting increasing attention, hardware restrictions are tight in current implementations. Intensive design exploration is therefore essential to match requirements, such as the problem scale and acceptable error rate, with potential designs to combine quantum computing and classical computing. The design decision made in this way is often fragile as it is sensitive to the problem scale as well as still evolving quantum services. We need continuous design decision, or adaptation and evolution, given changes in requirements or environments. In this paper, we present a framework for model-based engineering to support the continuous adaptation and evolution of quantum-classical hybrid systems. Modeling in our framework involves not only potential designs, but also rationale or evidence of design decision, which often requires simulation and experiments. This focus allows for tracing and analyzing whether the past decision is still valid or not, or whether there is uncertainty and we need further simulation and experiments. The usage of the framework is demonstrated with an example problem from steel manufacturing.

CCS CONCEPTS

• **Software and its engineering** → **Development frameworks and environments**; • **Computer systems organization** → **Quantum computing**.

KEYWORDS

Quantum Software Engineering, Hybrid Quantum Computing, Services Computing, DevOps, Self-Adaptive Systems, Models@run.time

ACM Reference Format:

Fuyuki Ishikawa and Shinobu Saito. 2024. Model-Based Framework for Continuous Adaptation and Evolution of Quantum-Classical Hybrid Systems. In *Companion Proceedings of the 8th International Conference on the Art, Science, and Engineering of Programming (Programming@Companion '24)*, March 11–15, 2024, Lund, Sweden. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3660829.3660849>

1 INTRODUCTION

Quantum computing has been drawing more and more attention because of its unique problem-solving abilities. It has been provided

as services by large cloud vendors such as Microsoft and Amazon Web Services as well as quantum-specialized companies such as D-Wave Systems. Engineering for quantum computing is currently receiving a wide range of attention.

Despite the expectation of scalability nature that may break the limitation of classical computers, hardware restrictions are tight in the current implementations. Specifically, fault-tolerant quantum computers are unlikely to be available in near future [18]. Algorithms based on heuristics are currently the primary focus of research for Noisy Intermediate-Scale Quantum (NISQ) computer. NISQ devices currently face significant limitations, including constraints on number of qubits (quantum bits) and the presence of noise processes that restrict circuit depth. Consequently, to work within these limitations, it becomes necessary to reduce the scale of problems, such as limiting the number of variables, while applying specialized algorithms to address these restrictions. Another promising approach is to use a specific type of quantum hardware called quantum annealing [10]. Quantum computers of this kind enable the handling of a larger quantity of qubits, typically numbering in the thousands, but only for a specific class of optimization problems.

Further enhancement is to decompose the problem so that a subproblem solved by quantum computing becomes smaller. As a simple example, suppose we have an optimization problem and one of the variables is $i \in [1, 10]$. We may handle i as a constant and run a smaller optimization for each $i = 1, 2, \dots, 10$ and then compare the obtained solutions to have the global optimal. We can apply more sophisticated decomposition such as Lagrangian decomposition. In this way, the design often becomes a workflow for combination of quantum computing and classical computing, leading to quantum-classical hybrid systems (QCHS). There is often trade-off between multiple quality attributes. For example, solving multiple smaller problems will allow for bypassing the hardware restrictions but require more time.

The above points implies that intensive design exploration is essential to match potential designs for QCHS with requirements, such as the problem scale and acceptable error rate. It is unclear what workflow we should apply until actually checking the hardware restrictions in the available quantum services. In other words, it is not like a waterfall or top-down process from abstract to concrete design as we have more uncertainty in the feasibility, which is often resolved by simulation and experiments such as estimation or measurement of qubit size and execution time [22].

Moreover, decisions made in this manner can be unstable and easily affected by changes in the problem size and the ongoing development of quantum services. As a result, we need continuously reoccurring design decision, or adaptation and evolution, given



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

Programming@Companion '24, March 11–15, 2024, Lund, Sweden
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0634-9/24/03
<https://doi.org/10.1145/3660829.3660849>

Time slot t	1	2	3	...	20	21
Machine 1	1	20	3	...	10	unused
Machine 2	idol	1	3	...	9	10

Table 1: Example Solution in Target JSP with $N = 20$ Products, $M = 2$ Machines, and $e = 1$ additional slot [2]

changes in requirements or environments. We may sometimes find that the past design decision is still valid given the past experiments. But we may find we don't have enough information to judge the feasibility in the updated requirements or environments. In any case, it is essential to trace and review the rationale or evidence of the past design decisions.

In this paper, we present a framework for model-based engineering for supporting continuous adaptation and evolution of QCHS. Modeling in our framework involves not only potential designs but also rationale or evidence of design decision. This focus allows for tracing and analyzing whether the past decision is still valid or not, or whether there is uncertainty and we need further simulation and experiments. We demonstrate use of the framework with an example problem from steel manufacturing [2, 27].

The remainder of the paper is organized as follows. We first describe an example application from steel manufacturing to illustrate potential designs for QCHS and then discuss relate work in Sections 2 and 3, respectively. We describe the proposed framework with concrete examples in Section 4. We conclude the paper with remarks for future directions in Section 5.

2 ILLUSTRATIVE EXAMPLE

2.1 Target Optimization Problem

We consider a job shop scheduling problem (JSP) for steel manufacturing provided in [2, 27]. Each of N products requires execution of jobs with M machines in the right order. Each job is executed in a fixed time slot. An example of scheduling solutions is shown in Table 1 for $N = 20, M = 2$. Each cell contains which product is handled in each time slot with each machine. For example, Product 1 is processed with Machine 1 at $t = 1$ and then Machine 2 at $t = 2$ in this solution. We may use $N + e$ time slots by including idol slots. The example solution uses 21 time slots with $e = 1$ with $N = 20$ by using an idol slot with Machine 2 at $t = 1$.

We consider the following constraints in this problem.

- C1 Each product is assigned to exactly one time slot in each machine.
- C2 N products are distributed to N consecutive time slots for each machine.
- C3 Each product must progress from Machine 1 to M in non-descending order.
- C4 Idol slots occur only at the beginning or end of the schedule.

Two optimization objectives are handled as follows.

- Minimize the penalty for early or late delivery. Each product has a due time. The penalty for a product is calculated by $c * t_d$ where c is a penalty constant and t_d is the number of time steps between the due time and the time when the last job for the product is completed. Different constant values

are used for early delivery and late delivery. The total penalty is the sum of the penalty values for each product.

- Minimize the cost for mismatch of product groups. Each product belongs to a product group. In each machine, we need to switch the machine configuration if the product groups of the adjacent products are different. The total cost is multiplication of a constant value and number of occurrences of the group mismatch.

The difficulty of the optimization problem comes from the trade-off between the two objectives, i.e., trying to collect products of the same group and process them together vs. processing products whose due time is coming.

In the quantum hardware, we often need to use binary variables to encode this problem, i.e., we denote x_{mpt} to represent whether Product p is processed with Machine m at time slot t . We then need at least MN^2 variables, which makes $2 * 20^2 = 800$ for $N = 20, M = 2$, optimistically by ignoring e additional slots. Indeed, it is possible to run optimization multiple times by fixing e as 1, 2, ... rather than considering e as a parameter to decrease the number of variables. Precise encoding is found in [2, 27].

2.2 Design Space

Different designs can be explored for the target problem. We discuss 4 possible designs below.

2.2.1 Design 1: Metaheuristics with Classical Computer. The target problem can be solved on a classical computer by making use of optimization techniques, for example, Simulated Annealing. The scalability of this design may be limited, which is why there is increasing expectations for quantum computing.

2.2.2 Design 2: VQA with Gate-based Quantum Computer. Gate-based quantum computers allow for solving many types of problems by combining gates to constitute a quantum circuit. However, the current implementation imposes tight constraints in the problem size.

Heuristics-based algorithms for noisy intermediate-scale quantum (NISQ) are dominant and are intensively investigated, as fault-tolerant quantum computers are unlikely to be available in the near future [18]. Current NISQ devices have serious restrictions, specifically, limited numbers of qubits (bits in the quantum setting) and noise processes that limit circuit depth. As a result, we need to limit the scale of the problem such as the number of variables while applying specific algorithms to deal with the restrictions. VQA (variational quantum algorithm) is a dominant strategy for such algorithms and is implemented in several detailed algorithms such as VQE, QAQA, VarQITE, and F-VQE [2, 6].

With this design, solving the problem with 23 variables was reported as the state-of-the-art scale, which was extracted as a sub-scheduling of the original full problem (with 800 variables at least) [2]. The probability of obtaining the optimal value decreases with a larger number of variables and was reported as 25% for 23 variables. The gate-based approach is generic and is attracting wide attention but may need to wait for further technical improvement.

2.2.3 Design 3-1: Quantum Annealing. There has been another type of quantum hardware called quantum annealing [10]. This type of quantum computers allows for dealing with a larger number of

qubits, currently thousands, for a specific class of problems called QUBO (quadratic unconstrained binary optimization). It should be noted the necessary number of qubits is larger than the number of variables in the mathematical representation. This is because connection between bits is limited in a specific topology such as a chimera graph, and we need to introduce additional qubits to represent the relation between variables.

Our target problem can be encoded as QUBO and could be efficiently solved for $N = 5, M = 2$ by fixing e as a constant. However, it was not solvable for $N = 8, M = 2$ due to the limitation in the number of variables [27].

2.2.4 Design 3-2: Quantum Annealing with Lagrangian Decomposition. To further increase the applicability of quantum annealing, we can apply Lagrangian decomposition (LDC) to divide the original optimization problem into a set of optimization problems for each machine. This makes the required number of variables from MN^2 to N^2 (solved M times).

For our target problem, it was reported this design enabled to solve the setting with $N = 8, M = 2$ [27]. A custom algorithm was applied to use quantum annealing as a sampler of approximate solutions rather than as an end-to-end optimizer. This design constitutes a workflow for QCHS that contains activities for both quantum and classical computing. The execution time increased compared with Design 3-1 due to the additional cycles in the case of $N = 5, M = 2$ where both designs can be applicable. This means there is trade-off between different approaches.

2.3 Motivation

We need to work on the design decision to identify the best design with the given requirements in any software systems. In the case of quantum systems, requirements on the problem size have a critical impact on the design decision in terms of feasibility, as seen in the aforementioned example. It is therefore necessary to provide specific support for design decision making to take this point into consideration.

In addition, the design decision may be fragile with two types of changes. First, the requirements about the size of the problem can evolve over time. In our example problem, the number of products and machines can be updated in monthly or quarterly planning, with changes in plant equipment, or in application to different plants. Second, the hardware constraints may be updated due to technical evolution in hardware or up/down-grade of service levels in hardware (cloud) providers. This point means that the task of decision decision may reoccur after the initial development.

In (classical) software engineering, this type of reoccurring design decision has been investigated in many paradigms. In Model-Driven Architecture (MDA) or Model-Driven Engineering (MDE), model transformation is investigated for efficient support of the reoccurring derivation of designs from requirements and environments [11]. In Service-Oriented Computing and Cloud Computing, implementation-level components are encapsulated as services, and reoccurring design decision is considered as problems of service selection, allocation, and composition [26]. For self-adaptive systems, the models@run.time approach makes use of development-time models at runtime so that the reoccurring design decision can be made automatically by the system [4].

In any case, the key is modeling requirements and designs, as well as mapping or matching between them. Therefore, we investigate a domain-specific framework to facilitate not only development but also continuous evolution and adaptation for QCHS.

It should be noted that feasibility checking or matching between requirements and quantum services often requires simulation or experiments. For example, execution time and error rate may be measured in experiments using the real implementations. It is even not obvious how many qubits are necessary, since encoding (embedding) is required from variables in the high-level mathematical problem to those in quantum implementations. Some providers of quantum services support the estimation of the necessary amount of such resources, e.g., in Microsoft Azure [22].

3 RELATED WORK

3.1 Paradigms in Classical Software Engineering

Model transformation has been investigated to efficiently generate concrete models from abstract models in MDE [11]. The MDE approach will support the derivation of models for design and implementation from those for requirements. This direction is orthogonal and complementary to our focus in this paper. We consider the evaluation, selection, and switching between multiple designs, which requires that each design be implemented and evaluated efficiently as in MDE.

In Service-Oriented Computing and Cloud Computing, there has been an active effort to select, allocate, and compose services [26]. The most relevant line of work is QoS-aware service selection [5]. Most studies in this line fixed an abstract workflow and searched for the best combination of concrete services allocated to activities in the workflow. We consider exploration of different abstract workflows as well as allocation of concrete services. The difference leads to a different procedure. Nevertheless, the established model of quality for individual services and a composite workflow is used as the basis for this work.

For self-adaptive systems, the models@run.time approach makes use of development-time models at runtime for automated adaptation to apply different designs [4]. One way of implementation is to model different alternatives to achieve the goals and, at runtime, switch between them by checking the conditions under which each alternative works well, e.g. [16]. We take the similar approach in a sense we explore potential designs and clarify evidence that serves as the rationale for selecting a specific design. We do not insist on full automation in this paper, though we will establish sufficiently sufficient and rigorous modeling to support that.

3.2 Design and Implementation Support in Quantum Software Engineering

Quantum software engineering has been attracting increasing attention [1]. There has been active effort for programming support as quantum computing is implemented in a different paradigm of hardware. It is becoming popular to use high-level programming languages and apply transformation or compilation to quantum circuits, for example, the Q# language [21]. There has been also active effort on testing of quantum programs or circuits [7]. Our

focus is more on high-level modeling and decision-making at the design level.

There have been several studies that extend traditional modeling languages for requirements and designs for QCHS. Requirements models were discussed in [28]. The extension of UML was discussed in [17] and involved the scale aspect and hardware restrictions as unique points. Workflow-based modeling was investigated in [25]. In those studies, specific points of QCHS have been involved to model quantum hardware concepts, hardware restrictions, and typical computation types. Our study may import such modeling languages while this paper focuses on recording and tracing of reoccurring decision making.

More and more support has been provided to accelerate implementation via model transformation or code generation. MDE in the quantum setting has been envisioned and discussed in [8]. The work in [23] implements a framework for transformation and deployment based on workflow models. The work in [19] envisions an automated framework for the process from problem definition to implementation. This direction is orthogonal to support an efficient horizontal process for implementing and evaluating one design, while we focus on the exploration of multiple designs in this paper.

Simulation or estimation techniques are also investigated to assess such quality attributes without experiments that run concrete implementations. Specifically, an estimator on Microsoft Azure has been available and tested with different types of quantum applications [3, 22]. It enables to estimate necessary amounts of resources such as number of qubits and processing time. This type of technique will facilitate each design decision while we focus on exploration of multiple designs in this paper.

In summary, the support for the evaluation or implementation of one design candidate has been actively investigated. Our work focuses on design decision over multiple design candidates on top of such support.

3.3 Design Exploration in Quantum Software Engineering

There have been case studies to compare multiple designs for specific problems, as discussed in Section 2 [2, 27]. The work in [12] discusses different ways of transforming the SAT (boolean satisfiability) problem to designs on top of quantum annealing. It was shown that a careful consideration is required to define and evaluate relevant quality attributes, specifically, scalability and accuracy. Our focus is how to facilitate continuous activities for such experience-based design exploration.

Automated selection of algorithms or designs has been investigated in several studies [14, 19, 20]. In those studies, predefined rules or a machine learning-based classifier was used for automated selection. Such mechanisms are effective when the design space is stable and we can build the rules or classifier from the past experience. We rather focus on situations in which we need an evaluation phase to compare different designs due to lack of sufficient previous experience or flexible updates. It is also notable that some studies only consider a narrow space of design, e.g., just selecting a popular quantum algorithm for the given problem class. We consider designs that span quantum and classical computing and also explore

different quality characteristics that affect selection at the service level.

Regarding continuous activities for QCHS, the development life-cycle for QCHS has been discussed in [24]. Although there was some mention on observation and analysis after deployment, concrete mechanisms for dealing with changes were not discussed. The DevOps paradigm in the context of NISQ hardware was discussed in [9]. The focus was primarily put on monitoring of performance in terms of error rate while we focus on more general quality attributes and investigate tracing mechanisms for reoccurring design decision.

In summary, this paper aims to develop the design exploration in those individual case studies into a generic framework by incorporating relevant tools for model transformation and resource estimation. We specifically envision a reoccurring design decision given different types of updates in environments and requirements, that is, adaptation and evolution of QCHS.

4 PROPOSED FRAMEWORK

4.1 Overview

Given the discussion in Section 2, we propose a modeling framework to support continuous design management for quantum-classical hybrid systems. Figure 1 illustrates what constitutes the model for the example problem.

The *requirements* part is at the top of the figure and defines the problem to be solved. In this specific example, the requirements define a single-objective optimization problem and thus accompany constraints as *hard goals* and weights over objectives as *soft goals*. The notable point is the *scale* attribute in *hard goal*, which defines the expected scope of parameters that affect the problem size and thus feasibility of potential designs.

The *abstract design* part defines candidates for architectural design and workflow, as illustrated in Section 2.2. The *concrete* design part defines allocation of concrete service instances for the workflow. This leveling of abstract and concrete workflows has been used for service-oriented computing with techniques such as the WS-BPEL workflow language [15] and QoS-aware service selection techniques [5]. The workflows in our setting include activities for quantum computing in addition to traditional computing and hardware restrictions tightly limit the feasible design space.

In this figure, design decisions are shown as check marks on both the abstract and concrete levels. Decisions are made by matching hard goals and soft goals with available design candidates. The decisions may be fragile when either of the problem scale or hardware restrictions is prone to be updated.

Figure 2 illustrates a situation when the problem scale is updated and the largest case of N becomes 8 from 5. This change invalidates the previous design decision, the design is then switched at the abstract level, and the concrete design is finally reselected. To detect the need for this switching, it is necessary to trace the rationale of the previous decision, specifically, specification of concrete services and experimental results used for comparing different designs. Supporting this kind of tracing on rationales or evidences is essential when either of requirements or hardware restrictions is likely to evolve.

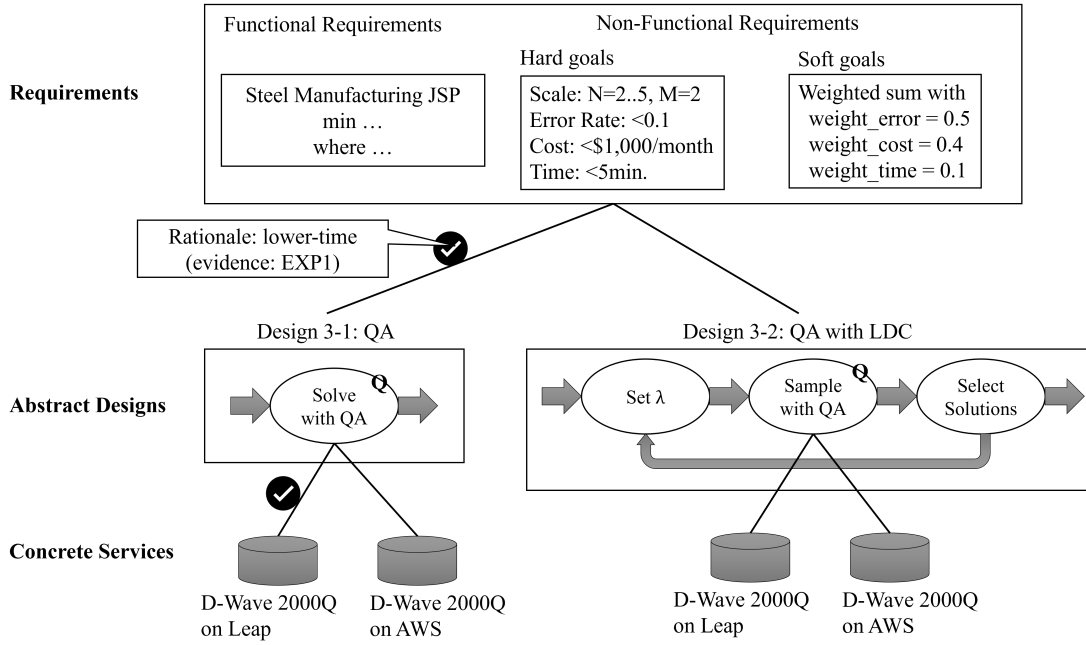


Figure 1: Modeling Elements for Design Decision in Target Problem

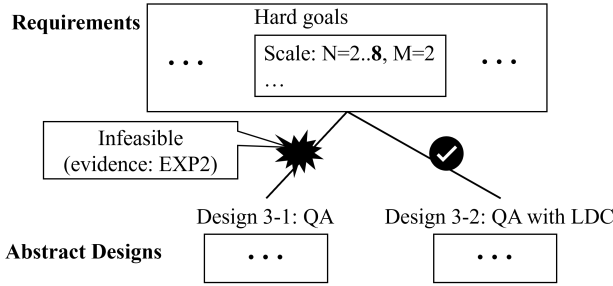


Figure 2: Updated Design Decision given Change in Requirements on Problem Scale

The iterated process illustrated above should be supported by models that represent the essence shown in the figure. This paradigm is just in analogy with disciplines in traditional software engineering. However, we need to think of the following natures rather than simply thinking of the waterfall, top-down process.

- Currently, hardware restrictions are strict, and we do not have sufficient choices for services at the concrete design level. In most cases, it is not possible to fix the abstract design without checking the feasibility or achievable quality levels with concrete service candidates.
- This assessment procedure may require experiments, not only matching requirements with published service specifications.

These properties have existed in some types of traditional software systems, but will be more dominant in QCHS.

Figure 3 describes the process supported by the proposed framework given the above points. The key step is 2. *Design Space Definition and Evaluation* after the *Requirements Analysis* step. In this step, each concrete design is constructed as a workflow and also evaluated in terms of whether it can satisfy the given requirements or not. For example, the required number of qubits to solve the target problem is estimated and then compared with hardware restrictions in the available services. The knowledge obtained in such evaluation is recorded as evidence. The process is executed again when necessary given the three types of changes depicted in the figure, that is, changes in requirements, available services, and the validity of evidence.

Below we describe detail of our modeling framework to support this kind of continuously iterated process.

4.2 Step 1: Requirements Analysis

The common nature of requirements analysis is the same for QCHS, e.g., involving diverse stakeholders to elicit requirements. The notable point is that quality attributes, or non-functional requirements, should cover key attributes that affect the design decision.

Specifically, the following attribute are thought as primary in the setting of QCHS. At this level of requirements specification, the attributes are represented in domain- or problem-level metrics, not in implementation-level metrics for quantum computing.

Scale The problem size that should be solved by the target QCHS or its scope for runtime configuration. This attribute is the primary factor that affects the feasibility in the current quantum services. In our example, this attribute is represented as a scope for each of the problem variables.

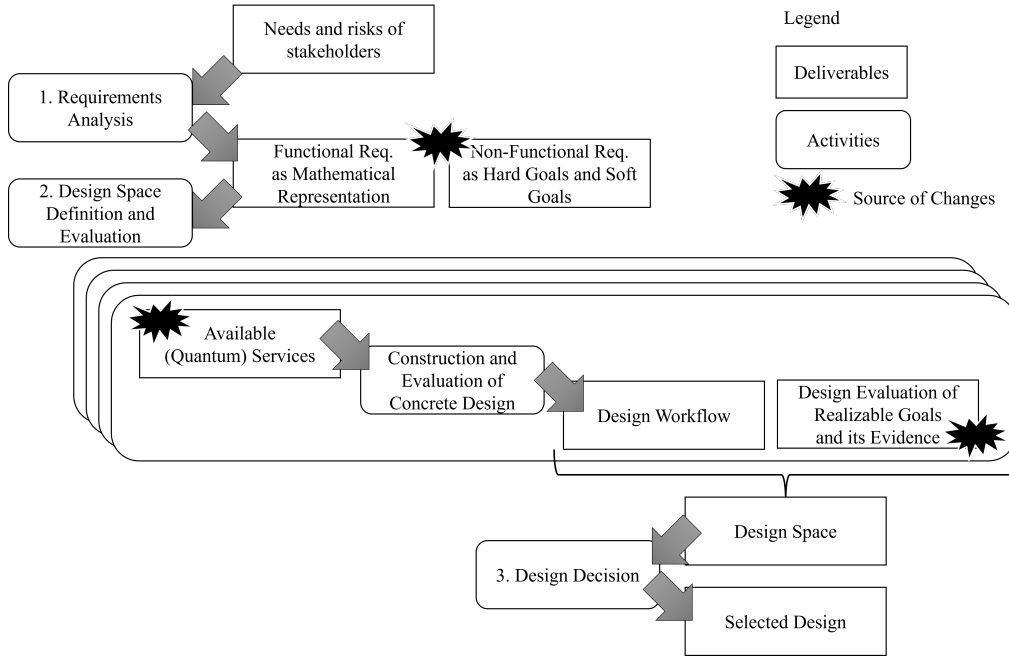


Figure 3: Process in Proposed Framework

Error Rate The extent of optimality in obtained solutions. This attribute is necessary as the current designs may apply heuristics rather than ensuring the optimal solution. For example, this attribute can be measured by experiments that compare the target design with brute-force optimization in a small-scale setting.

Execution Time The time required to complete the task to obtain the solution. This attribute also depends on the design and should be considered together with the error rate to evaluate the design.

We may consider other attributes common to QoS-aware service composition in the context of service-oriented computing and cloud computing [5]. For example, price or cost is a common attribute that is often in trade-off with the other attributes, e.g., pay more to use the latest hardware. Reliability and availability are also common, and we may use multiple services for the same functionality as backups to improve this quality attribute.

At this point, the feasibility of the requirements may be uncertain; that is, they may be too ideal to realize in the current QCHS. The feasibility is validated in the following step and the requirements may be adjusted given feedback from the following steps.

4.3 Step 2: Design Space Definition and Evaluation

In the next step, we investigate potential designs with available quantum services. As the hardware restrictions in quantum services are currently tight and the available choices are limited, it is not possible to fix an abstract design without checking its feasibility. So, we rather define the whole design space, not making one abstract design.

As we illustrate in Figure 1, each abstract design defines a workflow to form QCHS to solve the target problem. A concrete design is instantiated from an abstract design by allocating concrete services to each activity. Here, concrete services are distinguished by the quality attributes achieved. For example, different plans or instance types from the same provider should be distinguished.

It should be noted that concrete services may cover multiple activities of the workflow as service providers prepare services for hybrid solvers.

4.3.1 Evaluation of Each Concrete Design. Each design is evaluated by its quality values. It is often necessary to implement simulations or experiments to assess the values of quantum-specific attributes. The feasibility of the required scale can be checked by actually trying the embedding or by using simulation or estimation, e.g., [22]. Execution time may also be estimated but rather measured through experiments with error rate. Other quality values are aggregated as in the total price or total reliability based on specifications or service-level agreements (SLAs). This assessment is essential for activities that use quantum services. The assessment of quality values for classical services can be done in the established way rather than experiments.

In this way, each design is judged as satisfactory or not in terms of the hard goals, especially the problem scale, and quality values for the soft goals are computed to be compared with other designs. In this paper, we do not define a concrete algorithm for efficient filtering of infeasible designs or designs that are dominated in any quality attributes and thus not necessary to keep as candidates of the next steps.

4.3.2 Feasibility Validation of Requirements. The feasibility of requirements can be also judged by checking there is at least one

or some enough number of potential designs that satisfy the hard goals.

4.3.3 Recording Evidence for Potential Decision. It is essential to record what has been shown by the experiments conducted. In the example described in Section 2, the first experiment showed that Design 3-1 was applicable to the scale parameter $N = 5$, and then the second experiment showed that it was not applicable to $N = 8$. The knowledge obtained can be interpreted as the applicability of the design is:

[true for $N \leq 5$, uncertain for $N \in [6, 7]$, false for $N \geq 8$.

We can query this knowledge so that additional experiments can be avoided if possible, for example, when N becomes 4 or 9.

In addition, we may record the comparison results on each quality attribute. Suppose we found that Design 1 is better in scale but worse in execution time than Design 2, and then selected Design 1 by priority in the given requirements. Then, we keep the knowledge, for example, as a graph with designs as nodes and the relationships as edges:

Design 2 *dominatedBy(scale)* Design 1
Design 1 *dominatedBy(execution time)* Design 2.

We can traverse the graph to find designs with better or worse values on a specific quality attribute. For example, we can easily find Design 2 as an alternative if we need a better value for execution time. If the priority on a quality attribute gets lower, we can traverse the relationships inversely to make compromise in the attribute and have better quality in other attributes. We can also check the uncertainty: which pair of designs has not been compared in the past experiments.

4.4 Step 3: Design Decision

Design decision can be made given the feasible designs and their quality values. This step is straightforward given the whole design space enumerated in the previous step. For example, we may apply the weighted sum to integrate multiple soft goals into one single criterion to compare multiple designs.

4.5 Step 4: Monitoring, Adaptation, and Evolution

The design decision process should be repeated upon various types of changes. The rationale of the previous decisions, including the experimental results, should be queried and reused if possible.

In this paper, three types of update are considered, and the corresponding monitoring mechanisms are necessary.

First, requirements are updated given periodical updates of the business planning as well as changes in the application target. For our example, the number of products can be updated in quarterly manufacturing planning. Installation of the system to different plants will change the number of products and machines.

Second, available services can be updated especially for quantum services still under active technical progress. This change will increase quality values or relax hardware restrictions, such as the number of qubits.

Last, evaluation of quality values for design decision may turn out to be inaccurate due to limited scale of experiments or the

probabilistic nature of quantum computing. This potential gap can be detected by monitoring the actual quality values during operation, specifically for execution time. The evidence should be then updated to include the data collected at runtime. This may affect the validity of the previous design decision.

4.6 Metamodel

Figure 4 shows the metamodel of the framework that covers modeling elements described above. The type of *Functional Requirements* is limited to optimization in the figure, but can cover other types such as SAT solving [12] and machine learning [13]. We omit details on the general way to constitute a workflow by *Composite Activity* such as sequential composition and branch.

The metamodel is based on common concepts in service-oriented computing, specifically, QoS-aware service selection [5]. We put emphasis on modeling of results from *Quality Evaluation* and *Goal Assessment* to be tracked and reused if possible.

5 CONCLUSIONS AND FUTURE PERSPECTIVES

This paper has introduced a model-based engineering framework to facilitate the continuous adaptation and evolution of quantum-classical hybrid systems. Within this framework, modeling encompasses not just potential designs, but also the rationale and evidence behind them, often in the form of experiments. This emphasis enables tracking and analysis of the ongoing validity of past decisions and the identification of areas of uncertainty that may require additional simulations and experiments. To illustrate the practical utility of the framework, we have provided a demonstration using a problem scenario from the field of steel manufacturing.

Primary future work includes automation and further validation with other types of QCHS. In this paper, we have focused on defining the systematic approach that makes the manual process efficient for *Quantum DevOps* to investigate feedback cycles for QCHS. But it has also opened up the possibility for automation. Implementation of the fully automated edition for the presented framework will allow for applying advanced paradigms such as *Quantum Models@run.time* or *Automated Quantum Service Composition*. Demonstrating values of such paradigms is a promising direction. Meanwhile, we will work to apply the framework to other types of QCHS for generalization and validation, such as various designs for SAT solving [12] and machine learning [13].

REFERENCES

- [1] Shaikat Ali, Tao Yue, and Rui Abreu. 2022. When software engineering meets quantum computing. *Commun. ACM* 65, 4 (mar 2022), 84–88.
- [2] David Amaro, Matthias Rosenkranz, Nathan Fitzpatrick, Koji Hirano, and Mattia Fiorentini. 2022. A case study of variational quantum algorithms for a job shop scheduling problem. *EPJ Quantum Technology* 9, 5 (2022).
- [3] Michael E. Beverland, Prakash Murali, Matthias Troyer, Krysta M. Svore, Torsten Hoefler, Vadym Kliuchnikov, Guang Hao Low, Mathias Soeken, Aarthi Sundaram, and Alexander Vaschillo. 2022. Assessing requirements to scale to practical quantum advantage. <https://arxiv.org/abs/2211.07629>.
- [4] Gordon Blair, Nelly Bencomo, and Robert B. France. 2009. Models@ run.time. *Computer* 42, 10 (2009), 22–27.
- [5] Gerardo Canfora, Massimiliano Di Penta, Raffaele Esposito, and Maria Luisa Villani. 2005. An approach for QoS-aware service composition based on genetic algorithms. In *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation (GECCO '05)*. Association for Computing Machinery, New York, NY, USA, 1069–1075.

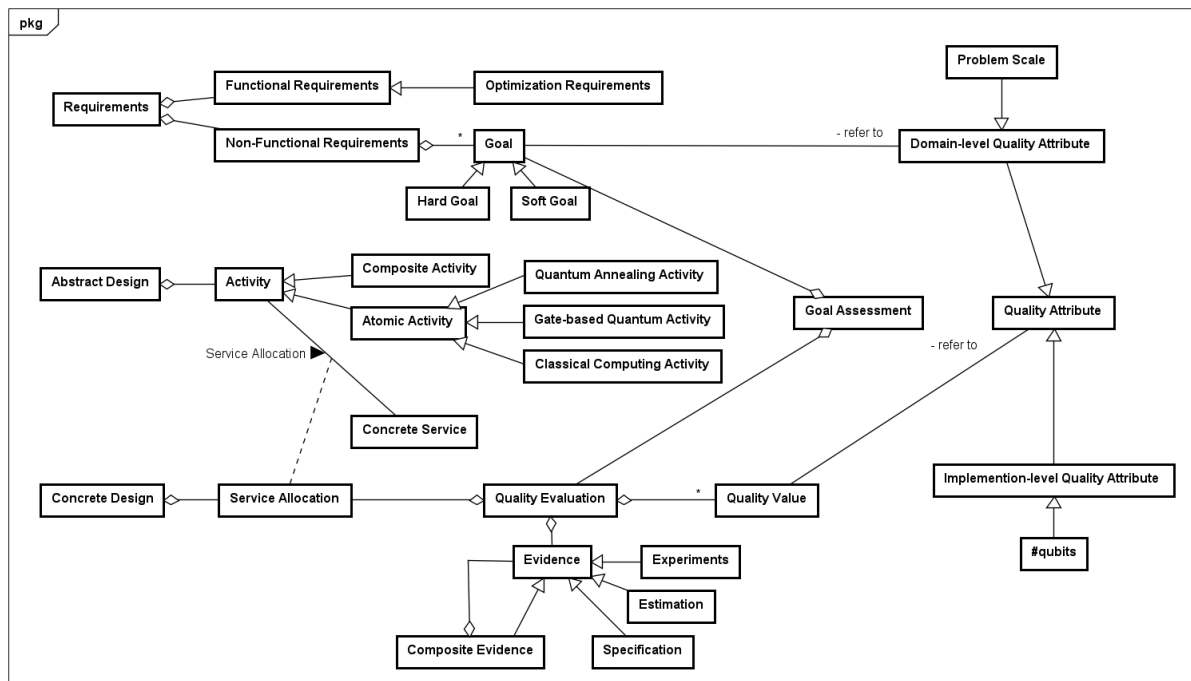


Figure 4: Metamodel

- [1] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. 2021. Variational Quantum Algorithms. *Nat Rev Phys* 3 (August 2021), 625–644.
- [7] Antonio García de la Barrera, Ignacio García-Rodríguez de Guzmán, Macario Polo, and Mario Piattini. 2023. Quantum software testing: State of the art. *Journal of Software: Evolution and Process* 35, 4 (2023), e2419. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2419>
- [8] Felix Gemeinhardt, Antonio Garmendia, and Manuel Wimmer. 2021. Towards Model-Driven Quantum Software Engineering. In *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. 13–15. <https://doi.org/10.1109/Q-SE52541.2021.00010>
- [9] Ilie-Daniel Gheorghe-Pop, Nikolay Tcholtchev, Tom Ritter, and Manfred Hauswirth. 2020. Quantum DevOps: Towards Reliable and Applicable NISQ Quantum Computing. In *2020 IEEE Globecom Workshops (GC Wkshps)*. 1–6.
- [10] Tadashi Kadowaki and Hidetoshi Nishimori. 1998. Quantum annealing in the transverse Ising model. *Phys. Rev. E* 58 (Nov 1998), 5355–5363. Issue 5.
- [11] Stuart Kent. 2002. Model Driven Engineering. In *Integrated Formal Methods*, Michael Butler, Luigia Petre, and Kaisa Sere (Eds.). 286–298.
- [12] Tom Krüger and Wolfgang Maurer. 2020. Quantum Annealing-Based Software Components: An Experimental Case Study with SAT Solving. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW'20)*. Association for Computing Machinery, New York, NY, USA, 445–450.
- [13] Ilya Sinayskiy, Maria Schuld, and Francesco Petruccione. 2015. An introduction to quantum machine learning. *Contemporary Physics* 56, 2 (2015), 172–185.
- [14] Charles Moussa, Henri Calandra, and Vedran Dunjko. 2020. To quantum or not to quantum: towards algorithm selection in near-term quantum optimization. *Quantum Science and Technology* 5, 4 (oct 2020), 044009. <https://doi.org/10.1088/2058-9565/abb8e5>
- [15] OASIS. 2007. Web Services Business Process Execution Language Version 2.0. <http://docs.oasis-open.org/wsbpel/2.0/wsbpel-v2.0.html>.
- [16] Xin Peng, Bihuan Chen, Yijun Yu, and Wenyun Zhao. 2012. Self-tuning of software systems through dynamic quality tradeoff and value-based feedback control loop. *Journal of Systems and Software* 85, 12 (2012), 2707–2719. <https://doi.org/10.1016/j.jss.2012.04.079> Self-Adaptive Systems.
- [17] Ricardo Pérez-Castillo and Mario Piattini. 2022. Design of classical-quantum systems with UML. *Computing* 104 (2022), 2375–2403.
- [18] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (August 2018), 79.
- [19] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. Towards an Automated Framework for Realizing Quantum Computing Solutions. arXiv:[2210.14928](https://arxiv.org/abs/2210.14928) [quant-ph]
- [20] Marie Salm, Johanna Barzen, Uwe Breitenbücher, Frank Leymann, Benjamin Weder, and Karoline Wild. 2020. The NISQ Analyzer: Automating the Selection of Quantum Computers for Quantum Algorithms. In *Service-Oriented Computing*, Shahram Dustdar (Ed.). Springer International Publishing, Cham, 66–85.
- [21] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (Vienna, Austria) (RWDSL2018)*. Association for Computing Machinery, New York, NY, USA, Article 7, 10 pages.
- [22] Wim van Dam, Mariia Mykhailova, and Mathias Soeken. 2023. Using Azure Quantum Resource Estimator for Assessing Performance of Fault Tolerant Quantum Computation. In *The SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 1414–1419.
- [23] Benjamin Weder, Johanna Barzen, and Frank Leymann. 2021. MODULO: Modeling, Transformation, and Deployment of Quantum Workflows. In *2021 IEEE 25th International Enterprise Distributed Object Computing Workshop (EDOCW)*. 341–344.
- [24] Benjamin Weder, Johanna Barzen, Frank Leymann, and Daniel Vietz. 2022. *Quantum Software Development Lifecycle*. Springer International Publishing, Cham, 61–83.
- [25] Benjamin Weder, Uwe Breitenbücher, Frank Leymann, and Karoline Wild. 2020. Integrating Quantum Computing into Workflow Modeling and Execution. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*. 279–291.
- [26] Yi Wei and M. Brian Blake. 2010. Service-Oriented Computing and Cloud Computing: Challenges and Opportunities. *IEEE Internet Computing* 14, 6 (2010), 72–75.
- [27] Kouki Yonaga, Masamichi Miyama, Masayuki Ohzeki, Koji Hirano, Hirokazu Kobayashi, and Tetsuaki Kurokawa. 2022. Quantum Optimization with Lagrangian Decomposition for Multiple-process Scheduling in Steel Manufacturing. *ISIJ International* 62, 9 (2022), 1874–1880.
- [28] Tao Yue, Shaikat Ali, and Paolo Arcaini. 2023. Towards Quantum Software Requirements Engineering. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 02. 161–164.

Received 2024-01-22; accepted 2024-02-11